

Inductive Moment Matching

	CIFAR-10	ImageNet-256×256				
Parameterization Setting						
Architecture	DDPM++	DiT-S	DiT-B	DiT-L	DiT-XL	DiT-XL
GFlops	21.28	6.06	23.01	80.71	118.64	118.64
Params (M)	55	33	130	458	675	675
$c_{\text{noise}}(t)$	$1000t$	$1000t$	$1000t$	$1000t$	$1000t$	$1000t$
2 nd time conditioning	s	s	s	s	s	$t - s$
Flow Trajectory	OT-FM	OT-FM	OT-FM	OT-FM	OT-FM	OT-FM
$g_\theta(\mathbf{x}_t, s, t)$	Simple-EDM	Euler-FM	Euler-FM	Euler-FM	Euler-FM	Euler-FM
σ_d	0.5	0.5	0.5	0.5	0.5	0.5
Training iter	400K	1.2M	1.2M	1.2M	1.2M	1.2M
Training Setting						
Dropout	0.2	0	0	0	0	0
Optimizer	RAdam	AdamW	AdamW	AdamW	AdamW	AdamW
Optimizer ϵ	10^{-8}	10^{-8}	10^{-8}	10^{-8}	10^{-8}	10^{-8}
β_1	0.9	0.9	0.9	0.9	0.9	0.9
β_2	0.999	0.999	0.999	0.999	0.999	0.999
Learning Rate	0.0001	0.0001	0.0001	0.0001	0.0001	0.0001
Weight Decay	0	0	0	0	0	0
Batch Size	4096	4096	4096	4096	4096	4096
M	4	4	4	4	4	4
Kernel	Laplace	Laplace	Laplace	Laplace	Laplace	Laplace
$r(s, t)$	$\max(s, \eta^{-1}(\eta_t - \frac{(\eta_{\max} - \eta_{\min})}{2^{15}}))$			$\max(s, \eta^{-1}(\eta_t - \frac{(\eta_{\max} - \eta_{\min})}{2^{12}}))$		
Minimum t , r gap	-	-	-	-	-	10^{-4}
$p(t)$	$\mathcal{U}(0.006, 0.994)$	$\mathcal{U}(0, 0.994)$	$\mathcal{U}(0, 0.994)$	$\mathcal{U}(0, 0.994)$	$\mathcal{U}(0, 0.994)$	$\mathcal{U}(0, 0.994)$
a	1	1	1	1	1	2
b	5	4	4	4	4	4
EMA Rate	0.9999	0.9999	0.9999	0.9999	0.9999	0.9999
Label Dropout	-	0.1	0.1	0.1	0.1	0.1
x -flip	True	False	False	False	False	False
EDM augment (Karras et al., 2022)	True	False	False	False	False	False
Inference Setting						
Sampler Type	Pushforward	Pushforward	Pushforward	Pushforward	Pushforward	Pushforward
Number of Steps	2	8	8	8	8	8
Schedule Type	$t_1 = 1.4$	Uniform	Uniform	Uniform	Uniform	Uniform
FID-50K ($w = 0$)	1.98	-	-	-	-	-
FID-50K ($w = 1.0$, i.e. no guidance)	-	42.28	26.02	9.33	7.25	6.69
FID-50K ($w = 1.5$)	-	20.36	9.69	2.80	2.13	1.99

Table 5. Experimental settings for different architectures and datasets.

ImageNet into latents without flipping, and calculate the channel-wise mean and std for normalization. We use Stable Diffusion VAE² and rescale the latents by channel mean $[0.86488, -0.27787343, 0.21616915, 0.3738409]$ and channel std $[4.85503674, 5.31922414, 3.93725398, 3.9870003]$. After this normalization transformation, we further multiply the latents by 0.5 so that the latents roughly have std 0.5. For DiT architecture of different sizes, we use the same hyperparameters for all experiments.

Choices for T and ϵ . By default assuming we are using mapping function $r(s, t)$ by constant decrement in η_t , we keep $\eta_{\max} \approx 160$. This implies that for time distribution of the form $\mathcal{U}(\epsilon, T)$, we set $T = 0.996$ for cosine diffusion and $T = 0.994$ for OT-FM. For ϵ , we set it differently for pixel-space and latent-space model. For pixel-space on CIFAR-10, we follow Nichol & Dhariwal (2021) and set ϵ to a small positive constant because pixel quantization makes smaller noise imperceptible. We find 0.006 to work well. For latent-space on ImageNet-256×256, we have no such intuition as in pixel-space. We simply set $\epsilon = 0$ in this case.

Exceptions occur when we ablate other choices of $r(s, t)$, e.g. constant decrement in λ_t in which case we set $\epsilon = 0.001$ to prevent $r(s, t)$ for being too close to t when t is small.

Injecting time s . The design for additionally injecting s can be categorized into 2 types – injecting s directly and injecting stride size $(t - s)$. In both cases, architectural designs exactly follow the time injection of t . We simply extract positional time embedding of s (or $t - s$) fed through 2-layer MLP (same as for t) before adding this new embedding to the embedding of t after MLP. The summed embedding is then fed through all the Transformer blocks as in standard DiT architecture.

²<https://huggingface.co/stabilityai/sd-vae-ft-mse>

Inductive Moment Matching

	10-step	16-step	32-step
FID w/ $w = 1.5$	1.98	1.90	1.89

Table 6. FID of ImageNet-256×256 beyond 8 steps presented in the main text. At 16 steps, IMM already outperforms VAR with 2B parameters (1.92 FID) and its performance saturates beyond that, with 32 steps performing marginally better.

	1-step	2-step	4-step	8-step
$a = 1$	7.97	4.01	2.61	2.13
$a = 2$	8.28	4.08	2.60	2.01

Table 7. Ablation of exponent a in the weighting function on ImageNet-256×256. We see that $a = 2$ excels at multi-step generation while lagging slightly behind in 1-step generation.

	1-step	2-step	4-step	8-step
TF32 w/ $a = 1$	7.97	4.01	2.61	2.13
FP16 w/ $a = 1$	8.73	4.54	3.03	2.38
FP16 w/ $a = 2$	8.05	3.99	2.51	1.99

Table 8. Ablation of lower precision training on ImageNet-256×256. For lower precision training, we employ both minimum gap $\Delta = 10^{-4}$ and $(t - r)$ conditioning.

Improved CT baseline. For ImageNet-256×256, we implement iCT baseline by using our improved parameterization with Simple-EDM and OT-FM schedule. We use the proposed pseudo-huber loss for training but find training often collapses using the same $r(s, t)$ schedule as ours. We carefully tune the gap to achieve reasonable performance without collapse and present our results in Table 2.

I.2. Inference Settings

Inference schedules. For all one-step inference, we directly start from $\epsilon \sim \mathcal{N}(0, \sigma_d^2 I)$ at time T to time ϵ through pushforward sampling. For all 2-step methods, we set the intermediate timestep t_1 such that $\eta_{t_1} = 1.4$; this choice is purely empirical which we find to work well. For $N \geq 4$ steps we explore two types of time schedules: (1) uniform decrement in t with $\eta_0 < \eta_1 \cdots < \eta_N$ where

$$t_i = T + \frac{N - i}{N}(\epsilon - T) \quad (117)$$

and (2) EDM (Karras et al., 2022) time schedule. EDM schedule specifies $\eta_0 < \eta_1 \cdots < \eta_N$ where

$$\eta_i = \left(\eta_{\max}^{\frac{1}{\rho}} + \frac{N - i}{N}(\eta_{\min}^{\frac{1}{\rho}} - \eta_{\max}^{\frac{1}{\rho}}) \right)^{\rho} \quad \text{and} \quad \rho = 7 \quad (118)$$

We slightly modify the schedule so that $\eta_0 = \eta_{\min}$ is the endpoint instead of $\eta_1 = \eta_{\min}$ and $\eta_0 = 0$ as originally proposed, since our η_0 can be set to 0 without numerical issue.

We also specify the time schedule type used for our best runs in Table 5 and their results.

I.3. Scaling Settings

Model GFLOPs. We reuse numbers from DiT (Peebles & Xie, 2023) for each model architecture.

Training compute. Following Peebles & Xie (2023), we use the formula model GFLOPs · batch size · training steps · 4 for training compute where, different from DiT, we have constant 4 because for each iteration we have 2 forward pass and 1 backward pass, which is estimated as twice the forward compute.

Inference compute. We calculate inference compute via model GFLOPs · number of steps.

I.4. Scaling Beyond 8 Steps

We investigate when the performance saturates for ImageNet-256×256 in Table 6. We see continued improvement beyond 8 steps and at 16 steps our method already outperforms VAR with 2B parameters (1.92 FID).

I.5. Ablation on exponent a

We compare the performance between $a = 1$ and $a = 2$ on full DiT-XL architecture in Table 7, which shows how a affects results of different sampling steps. We observe that $a = 2$ causes slightly higher 1-step sampling FID but outperforms $a = 1$ in the multi-step regime.

I.6. Caveats for Lower-Precision Training

For all experiments, we follow the original works (Song et al., 2020b; Peebles & Xie, 2023) and use the default TF32 precision for training and evaluation. When switching to lower precision such as FP16, we find that our mapping function, *i.e.* constant decrement in η_t , can cause indistinguishable time embedding after some MLP layers when t is large. To mitigate this issue, we simply impose a minimum gap Δ between any t and r , for example, $\Delta = 10^{-4}$. Our resulting mapping function becomes

$$r(s, t) = \max \left(s, \min \left(t - \Delta, \eta^{-1}(\eta(t) - \epsilon) \right) \right)$$

Optionally, we can also increase distinguishability between nearby time-steps inside the network by injecting $(r - s)$ instead of s as our second time condition. We use this as default for FP16 training. With these simple changes, we observe minimal impact on generation performance.

Lastly, if training from scratch with lower precision, we recommend FP16 instead of BF16 because of higher precision that is needed to distinguish between nearby t and r .

We show results in Table 8. For FP16, $a = 1$ causes slight performance degradation because of the small gap issue at large t . This is effectively resolved by $a = 2$ which downweights losses at large t by focusing on smaller t instead. At lower precision, while not necessary, $a = 2$ is an effective solution to achieve good performance that matches or even surpasses that of TF32.

J. Additional Visualization

We present additional visualization results in the following page.